

JUSTIFICATION

We have chosen **Layered Architecture** for the **Book Club Reading Challenge & Discussion Portal** System.

Architecture Choice - Layered Architecture is where the system is organised into hierarchical layers and each layer has its specific role and interacts only with adjacent layers.

In this particular system, there are three layers - Presentation {for the Dashboard of Club Member and Discussion Lead interactions}, Business {for managing the services like monthly polls, reading goals, discussions and spoiler filtering} and Data {for storage}. Thus, there is a clear separation of concerns and thus, it suits the system.

Reasons -

- i) *Domain Isolation*: There are various different and unrelated functions like spoiler content viewing, and monthly polls under one common system. The system is able to handle several distinct workflows such as FR-001(spoiler blur), FR-002(monthly goals) and FR-003 (monthly poll). By isolating them into separate components, we can ensure that the rules for validation of these workflows can be decoupled and modular.
- ii) *Reduced Overhead*: The requirements of the portal are various and different, such as reading progress, discussion threads, handling polls, and voting. By using layered architecture, we can ensure that there is a consistent structure for different functionalities which is centralised. Thus, more features can be added without breaking the system. This also makes the maintenance easy, as compared to microservices architecture which would carry greater overhead for network and maintaining data consistency, whereas client-server architecture would be weaker at distinguishing the various responsibilities of the components.

Security Advantage -

By using Layered Architecture, we can ensure that the Data Layer has controlled access via the Business layer, instead of the user interface directly accessing it. Authentication and session verification can be performed before the database access. For instance, the Poll and Voting component can be used in order to validate the active member session and check the vote state before registering a cast vote, thus complying with the NFR-001 (prevent duplicate vote). The Spoiler Filter component can be used to ensure the spoiler's visibility is only when required, instead of handing this over directly to the client display logic completely.

Performance Benefit -

Discrete functionalities ensure that the requests made by the dashboard are processed through focused components instead of piling all functionalities inside a single module. Because the frontend layer only interacts with relevant business components which all use one single interface for accessing the data, the processing is reduced. As there are no duplicate database interfaces, the querying becomes more efficient, and caching might also be supported inside the suitable layer. This will ensure that the NFR-002 (load dashboard under 5 sec.) can be met under normal conditions.